

Script de Demo — Motor de Recomendación Neo4j (10 min)

[0:00 – 0:45] Introducción al caso de uso

“Nuestro proyecto es un motor de recomendación de series de streaming modelado como grafo en Neo4j. El modelo incluye 7 tipos de nodos: **Usuario, Serie, Actor, Género, Plataforma, Estudio y Reseña**. Hay más de 10 tipos de relaciones como VIO, LE_GUSTA, ACTUA_EN, DIRIGE, TRANSMITE, PERTENECE_A, SIMILAR_A, EN_LISTA, SIGUE_A, ESCRIBIO, PRODUJO y SOBRE. Esto nos permite hacer consultas de recomendación basadas en comportamiento real del usuario.”

Abrir Neo4j AuraDB y mostrar el grafo visualmente — nodos de distintos colores, muchas relaciones.

“El grafo es **conexo** — todos los nodos están relacionados entre sí.”

[0:45 – 1:30] Set de datos

Abrir la consola de Neo4j o el panel admin de la app:

```
MATCH (n) RETURN count(n)
```

“Tenemos más de 5000 nodos cargados en la base de datos.”

“Los datos fueron cargados desde archivos CSV. Voy a mostrarlo.”

Ir al panel **Admin > Cargar CSV** en el frontend → subir un CSV de prueba o ejecutar la carga.

“El sistema lee el CSV y crea nodos y relaciones con todas sus propiedades en Neo4j.”

[1:30 – 2:15] Tipos de datos y propiedades

Abrir Neo4j y correr:

```
MATCH (s:Serie) RETURN s LIMIT 1
```

“Una Serie tiene: **titulo** (String), **temporadas** (Integer), **calificacion** (Float), **activo** (Boolean), **generos** (List), **fechaEstreno** (Date). Cubrimos todos los tipos de datos requeridos.”

[2:15 – 3:15] Crear nodos

Nodo con 1 label — ir al frontend, panel Admin > Series:

“Creo una serie nueva con 1 label: **Serie**.”

Llenar el formulario y guardar.

Nodo con 2+ labels — ir a Actores > agregar director:

“Un Actor puede también ser Director. Al marcarlo como director, el nodo recibe 2 labels: **Actor** y **Director**.”

Hacer click en “Agregar como director” sobre un actor existente.

Nodo con 5+ propiedades — mostrar la creación de serie con todos los campos:

“Al crear una serie configuramos: título, sinopsis, año, temporadas, calificación, activo, fecha de estreno — más de 5 propiedades.”

[3:15 – 4:30] Visualización y filtrado de nodos

Ir al panel de series con filtros:

“Puedo consultar un solo nodo por ID...”

Buscar `ser_0001`.

“...o muchos nodos con filtros. Por ejemplo, series del género Acción con más de 3 temporadas.”

Aplicar filtros y mostrar resultados.

“Y también consultas agregadas:”

```
MATCH (s:Serie)
```

```
RETURN avg(s.calificacion) AS promedio, count(s) AS total
```

[4:30 – 5:45] Gestión de propiedades en nodos

Agregar propiedad a 1 nodo — editar una serie desde el frontend, agregar un campo nuevo.

“Agrego la propiedad **presupuesto** a esta serie.”

Actualizar 1 nodo — editar calificación de la misma serie.

Actualizar múltiples nodos — desde el endpoint masivo:

“Puedo actualizar la propiedad **activo** en todas las series de una plataforma al mismo tiempo.”

Usar la función de actualización masiva (PATCH `/series/masivo`).

Eliminar propiedad de 1 nodo — eliminar **presupuesto** de la serie.

Eliminar propiedad de múltiples nodos — mostrar endpoint masivo de eliminación de propiedades.

[5:45 – 6:45] Creación y gestión de relaciones

Crear relación con 3+ propiedades — ir a Relaciones, relacionar un actor con una serie:

“Creo la relación ACTUA_EN entre un actor y una serie. Incluye: **personaje** (String), **temporadas** (List), **protagonista** (Boolean) — 3 propiedades.”

Actualizar relación — cambiar el personaje o agregar una propiedad.

Actualizar múltiples relaciones — actualizar el progreso de visualización de todas las series de un usuario:

“Actualizo la relación VIO para todas las series de un usuario en un PATCH masivo.”

Eliminar propiedad de relación y de múltiples relaciones — demo rápido desde el frontend.

[6:45 – 7:30] Eliminación de nodos y relaciones

Eliminar 1 relación — quitar el like de un usuario a una serie:

“Elimino la relación LE_GUSTA entre este usuario y esta serie.”

Eliminar múltiples relaciones — quitar todas las series de la lista de un usuario (DELETE masivo).

Eliminar 1 nodo — eliminar la serie de prueba que creamos antes.

Eliminar múltiples nodos — desde DELETE `/series/masivo` con una lista de IDs.

[7:30 – 9:30] Consultas Cypher (2 por integrante)

Consulta 1 — Recomendaciones basadas en historial

```
MATCH (u:Usuario {id: 'usr_0001'})-[:VIO]->(s:Serie)-[:PERTENECE_A]->(g:Genero)
MATCH (rec:Serie)-[:PERTENECE_A]->(g)
WHERE NOT (u)-[:VIO]->(rec)
RETURN rec.titulo, count(g) AS coincidencias
ORDER BY coincidencias DESC LIMIT 10
```

“Series que el usuario no ha visto pero que comparten géneros con lo que sí vio.”

Consulta 2 — Top series por calificación y género

```
MATCH (s:Serie)-[:PERTENECE_A]->(g:Genero)
RETURN g.nombre, s.titulo, s.calificacion
ORDER BY g.nombre, s.calificacion DESC
```

Consulta 3 — Usuarios más influyentes (más seguidores)

```
MATCH (u:Usuario)<-[:SIGUE_A]-(seguidor)
RETURN u.nombre, count(seguidor) AS seguidores
ORDER BY seguidores DESC LIMIT 10
```

Consulta 4 — Actores que también dirigen

```
MATCH (a:Actor:Director)
OPTIONAL MATCH (a)-[:ACTUA_EN]->(s1:Serie)
OPTIONAL MATCH (a)-[:DIRIGE]->(s2:Serie)
RETURN a.nombre, count(DISTINCT s1) AS series_actuadas, count(DISTINCT s2) AS series_dirigi
```

[9:30 – 10:00] Cierre

“En resumen: 7 labels de nodos, 12 tipos de relaciones, 5000+ nodos, grafo conexo, CRUD completo de nodos y relaciones con operaciones individuales y masivas, carga CSV, filtros, agregaciones y un sistema de recomendaciones funcional conectado al frontend en React.”